



Introduction to Data Modeling in Power BI

Section 1

Relationships in Power BI

One-to-Many · Many-to-Many · One-to-One

Data modeling is the foundation of every reliable Power BI report. Before you build a single chart or write a single formula, you must ensure that your data is connected correctly. This section introduces the concept of relationships — the links between tables that allow Power BI to answer questions that span multiple datasets.

1.1 What is a Relationship?

Relationship

A defined connection between two tables in a data model, based on a common column (called a key) that appears in both tables. Relationships allow Power BI to correctly combine and filter data across multiple tables when building reports.

To understand why relationships are necessary, consider what happens without them. If you have a table of Sales transactions and a separate table of Product details, Power BI has no way of knowing that the 'Product ID' in the Sales table corresponds to the 'Product ID' in the Products table — unless you explicitly define that connection. A relationship is that explicit connection.



Analogy — Understanding Relationships

Think of a hospital's administration system. There are three separate registers maintained:

- A Patient Register — containing Patient ID, Name, Age, and Ward.
- An Appointments Register — containing Appointment ID, Patient ID, Doctor ID, and Date.
- A Doctors Register — containing Doctor ID, Name, Specialization, and Department.

Each register is a separate table. To answer the question 'Which doctor treated Patient Arjun Sharma on 14 March?', you need to look up Arjun's Patient ID in the Appointments register, then use the Doctor ID from that appointment to look up the doctor's name in the Doctors register.

This cross-table lookup is only possible because Patient ID and Doctor ID are the common keys that link the three registers together. In Power BI, defining these links between tables is called creating a relationship.

1.2 Types of Relationships

Not all relationships between tables work the same way. The type of relationship depends on how the values in the key column appear in each table. Power BI supports three types: One-to-Many, Many-to-Many, and One-to-One.

Type 1 — One-to-Many (1:*)

One-to-Many (1:*)

A relationship where one row in the first table (the 'one' side) can match multiple rows in the second table (the 'many' side). This is the most common relationship type in data modeling and is used in the majority of Power BI reports.

In a One-to-Many relationship, the key column on the 'one' side contains unique values — no duplicates. The key column on the 'many' side can contain the same value multiple times, because multiple records can refer to the same single item.



Professional Example — One-to-Many

Consider a retail company with the following two tables:

Customers Table (One side):		Orders Table (Many side):		
Customer ID	Customer Name	Order ID	Customer ID	Amount
C001	Meera Patel	O1001	C001	4,500
C002	Rahul Desai	O1002	C001	1,200
C003	Anita Joshi	O1003	C002	8,750
		O1004	C003	3,300

O1005	C001	2,100
Customer C001 (Meera Patel) appears three times in the Orders table — she has placed three orders. But she appears only once in the Customers table. This is a One-to-Many relationship: one customer record corresponds to many order records.		

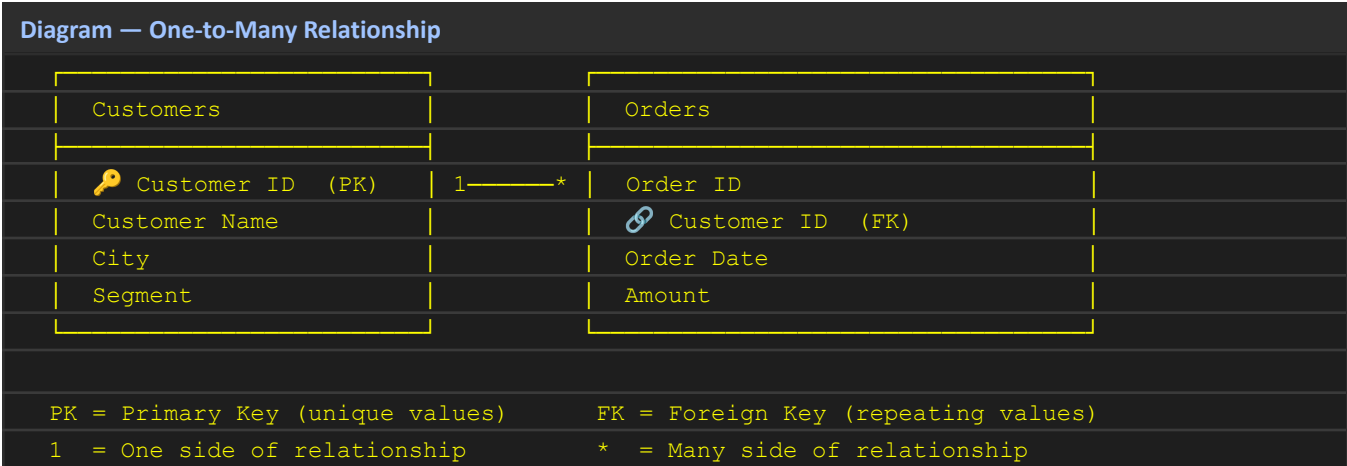


Diagram 1.1 — One-to-Many relationship between Customers and Orders

Type 2 — Many-to-Many (*:*)

Many-to-Many (*:*)	A relationship where multiple rows in the first table can match multiple rows in the second table. Neither table's key column contains uniquely distinct values — the same value can appear more than once in both tables.
---------------------------	--

Many-to-Many relationships are more complex and require careful handling. In earlier versions of Power BI, they had to be resolved using a bridge (junction) table. In modern Power BI, they can be defined directly, but they carry specific limitations that you must be aware of.

Professional Example — Many-to-Many			
A training institute has two tables:			
Students Table:		Courses Table:	
Student ID	Student Name	Course ID	Course Name
S001	Priya Menon	CR01	Data Analytics
S002	Karan Shah	CR02	Excel Advanced
S003	Divya Rao	CR03	Power BI

Enrollments Table (what actually exists):	
Student ID	Course ID
S001	CR01
S001	CR03
S002	CR01
S002	CR02
S003	CR01
S003	CR03
Priya is enrolled in 2 courses. Course CR01 has 3 students. Neither table has a unique key that maps 1-to-1 with the other. This is a Many-to-Many relationship. The Enrollments table acts as a bridge between Students and Courses.	

⚠ Important — Handling Many-to-Many Correctly

Direct Many-to-Many relationships in Power BI can lead to unexpected results in calculations if not set up correctly. The recommended approach is to introduce a bridge table (also called a junction or linking table) that breaks the Many-to-Many into two One-to-Many relationships.

In the example above, the Enrollments table serves as the bridge: Students has a One-to-Many with Enrollments, and Courses also has a One-to-Many with Enrollments.

Type 3 — One-to-One (1:1)

One-to-One (1:1)

A relationship where each row in the first table corresponds to exactly one row in the second table, and vice versa. Both key columns contain unique, non-repeating values.

One-to-One relationships are the least common type in data modeling. They typically occur when a single entity's data has been split across two tables — often for organizational, security, or performance reasons.

📋 Professional Example — One-to-One

A company has two tables:

Employees Table:

Employee_HR_Details Table:

Employee ID | Name | Department Employee ID | Salary | Bank Account

E001		Aditya Kumar		Sales	E001		65,000		***4521
E002		Sonal Mehta		Finance	E002		72,000		***8834
E003		Rohan Nair		IT	E003		80,000		***2290
Every employee has exactly one HR record, and every HR record belongs to exactly one employee. This 1:1 split is done deliberately — the sensitive salary and bank details are kept in a separate restricted table for security purposes.									

Relationship Types — Summary

Type	Symbol	Key on Side A	Key on Side B	Most Common Use
One-to-Many	1 : *	Unique values only	Repeated values allowed	Dimension table linked to Fact table (e.g., Products to Sales)
Many-to-Many	* : *	Repeated values	Repeated values	Students & Courses, Employees & Projects
One-to-One	1 : 1	Unique values only	Unique values only	Split entity tables (e.g., Employees & HR Details)

ETHANS
CLOUD & AI ACADEMY



Section 2

Cardinality & Cross Filter Direction

Controlling How Filters Flow Between Tables

When you define a relationship between two tables in Power BI, you must set two key properties: Cardinality and Cross Filter Direction. Together, these two settings determine how data is matched between tables and how filter selections in one table affect the data displayed in another.

2.1 Cardinality

Cardinality

The property of a relationship that describes how many rows in one table can match how many rows in the other table. In Power BI, cardinality is displayed as symbols — '1' for the unique side and '*' (asterisk) for the many side — on the relationship line in Model View.

Cardinality is automatically detected by Power BI based on whether the key column contains unique values or repeated values. However, it is important to understand how to verify and manually adjust this setting when the automatic detection is incorrect.

2.2 Cross Filter Direction

Cross Filter Direction

A relationship setting that controls which direction a filter selection travels between two connected tables. When you click or select a value in one table's visual, the cross filter direction determines whether that selection also filters the data in the other connected table.

This is one of the most important — and most misunderstood — settings in Power BI. Getting cross filter direction wrong can cause your visuals to show incorrect totals, unfiltered data, or no data at all. Understanding it clearly will save you significant troubleshooting time.

Single Direction (Default)

In Single direction, the filter flows in only one direction — from the 'one' side table to the 'many' side table. This means selections in the dimension table (the 'one' side) will filter the fact table (the 'many' side), but not the other way around.



Analogy — Single Filter Direction

Think of a water pipe. Water flows in only one direction — from the tank (the 'one' side — the Products table) to the taps (the 'many' side — the Sales table).
If you select 'Electronics' in a Product Category filter, it will filter the Sales table to show only electronics sales. However, selecting a specific sale in the Sales table will NOT filter the Products table to highlight only the products sold.
Single direction is the default and recommended setting for most relationships because it is predictable and prevents unintended filter interactions.

Both Directions (Bidirectional)

In Both directions (bidirectional), filters flow in both directions across the relationship. A selection in either table will filter the other. This is more powerful but also more dangerous — it can produce incorrect calculations if not applied carefully.

 Analogy — Bidirectional Filter Direction
Imagine a two-way road between a Products table and a Sales table. Traffic (filters) can flow in both directions.
If you select a specific product, it filters the sales for that product. If you select a specific sales region, it filters which products appear as having been sold in that region.
This is useful for specific scenarios — such as when you want a slicer on the 'many' side to filter the 'one' side — but it must be used with caution, as it can create ambiguous filter paths in complex models.

Single Direction	Both Directions (Bidirectional)
Filter flows from 'one' side to 'many' side only.	Filter flows in both directions between tables.
Default setting — recommended for most models.	Non-default — use only when specifically required.
Predictable and safe for complex models.	Can cause unexpected results in models with multiple tables.
Best for: Standard dimension-to-fact filtering.	Best for: Specific reporting scenarios like counting distinct values across tables.

As a best practice, always start with Single direction cross filtering. Only switch to Both directions when you have a specific, well-understood need for it. Using Both directions everywhere in a model is a common beginner mistake that leads to incorrect totals and circular dependency errors.



Section 3

Star Schema vs Snowflake Schema

Designing Your Data Model for Performance and Clarity

When building a data model in Power BI, the way you arrange and connect your tables follows a design pattern called a schema. The two most widely recognized schema designs in business intelligence are the Star Schema and the Snowflake Schema. Understanding both is essential to building models that are efficient, easy to maintain, and produce accurate results.

3.1 Key Terminology: Fact Tables and Dimension Tables

Before comparing schemas, you need to understand two fundamental types of tables that form the building blocks of any data model.

Fact Table	The central table in a data model that stores measurable, quantitative data — the actual business events or transactions. Examples: Sales transactions, Purchase orders, Production records, Website visits. Fact tables typically have a large number of rows and contain numeric columns (amounts, quantities, counts) alongside foreign keys that link to dimension tables.
CLOUD & AI ACADEMY	
Dimension Table	A supporting table that contains descriptive, categorical information about the entities involved in a business event. Examples: Products, Customers, Employees, Locations, Date. Dimension tables have fewer rows than fact tables and provide the context for analyzing the facts.

🔍 Analogy — Fact vs Dimension Tables	
Think of a university's examination system:	
Fact Table — Exam Results:	This is the event. Each row is one student
Student ID Subject ID Date ID Marks	taking one exam on one date. It has many
S001 SUB-MA D20250315 82	rows and stores the actual result.
S001 SUB-PH D20250316 74	
S002 SUB-MA D20250315 91	

Dimension Tables — The context:	These describe WHO (student), WHAT (subject),
Students: S001 = Riya Kapoor, Year 2	and WHEN (date) the exam occurred.
Subjects: SUB-MA = Mathematics	
Date: D20250315 = 15 March 2025	
The Fact table is lean — it stores numbers and keys. The Dimension tables provide the descriptive labels that make those numbers meaningful.	

3.2 The Star Schema

Star Schema	A data model design where a single central Fact table is directly connected to multiple Dimension tables through One-to-Many relationships. When drawn visually, the diagram looks like a star — the Fact table at the center, Dimension tables radiating outward like points of a star.
-------------	--

The Star Schema is the recommended and most widely used design pattern in Power BI. Microsoft explicitly recommends it because it delivers the best performance, simplest DAX formulas, and the most predictable filter behavior.

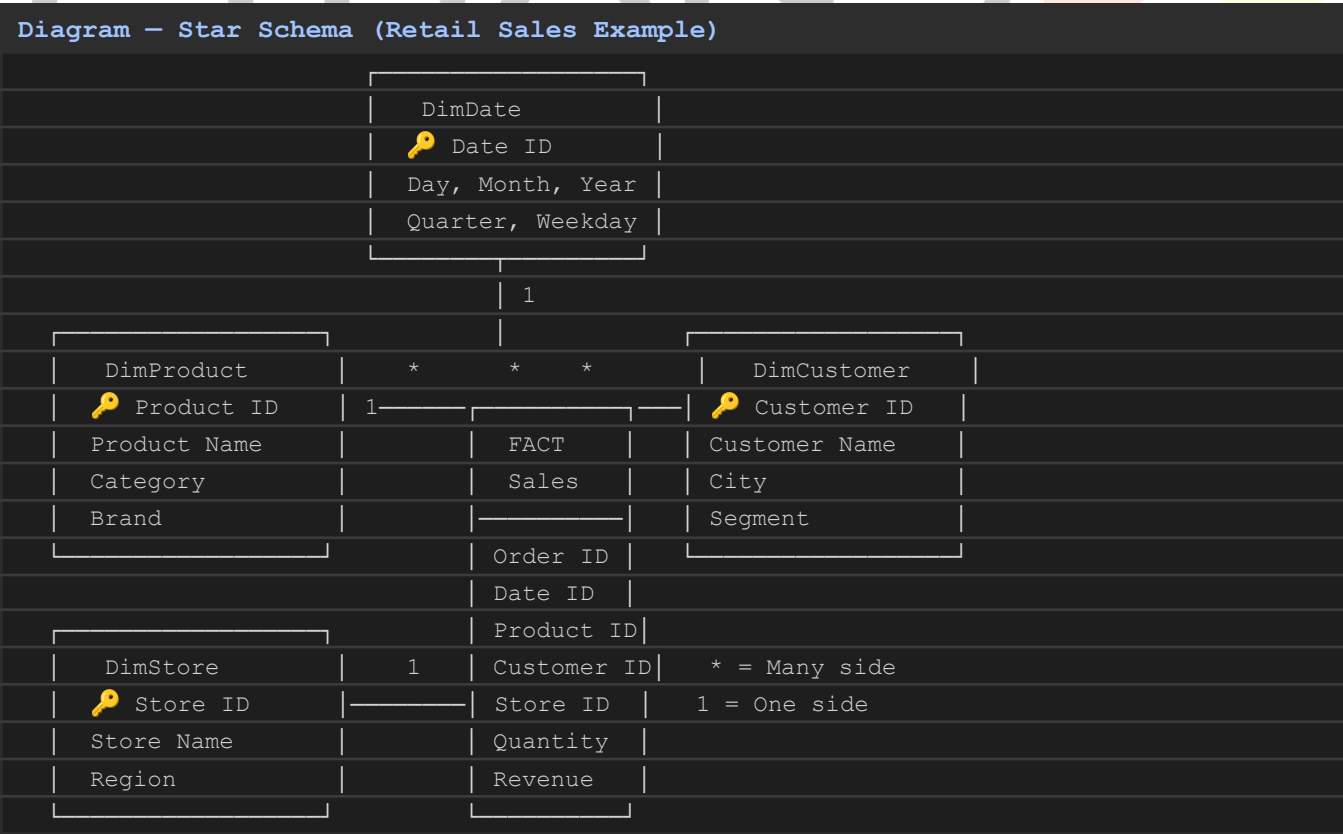


Diagram 3.1 — Star Schema with Sales Fact table and four Dimension tables

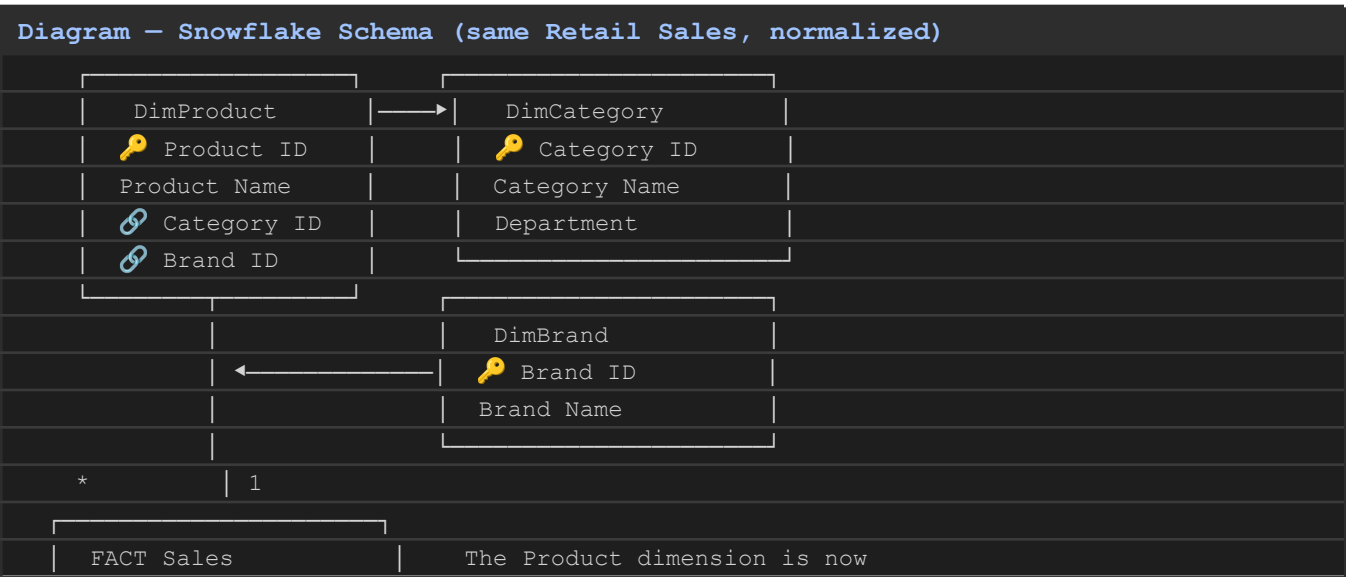
Why the Star Schema is Preferred in Power BI

Advantage	Explanation
Fastest query performance	Power BI's DAX engine (VertiPaq) is optimized for star schema designs. Fewer joins between tables means faster calculations and report rendering.
Simpler DAX formulas	With dimension tables directly connected to the fact table, filter context in DAX is predictable and easy to manage.
Easy to understand	The visual layout is intuitive — business users and developers can both understand the model at a glance.
Recommended by Microsoft	Power BI documentation explicitly advises star schema as the target design for production-grade data models.

3.3 The Snowflake Schema

Snowflake Schema	A variation of the Star Schema where Dimension tables are further broken down (normalized) into additional sub-dimension tables. Instead of one flat Dimension table, the descriptive data is split across multiple related tables, creating a branching structure that resembles a snowflake when drawn visually.
-------------------------	--

The Snowflake Schema typically comes from normalized relational databases — such as SQL Server or Oracle — where data engineers have broken tables down to reduce data redundancy. When you connect Power BI directly to such a database, you may receive a snowflake structure that then needs to be handled correctly.



Order ID	'normalized' – split into
Product ID	Product + Category + Brand.
...	More joins = slower queries.

Diagram 3.2 — Snowflake Schema showing a normalized Product dimension

Star Schema vs Snowflake Schema — Comparison

Star Schema	Snowflake Schema
Dimension tables are flat (denormalized).	Dimension tables are further split into sub-dimensions.
Fewer tables, simpler structure.	More tables, more complex structure.
Faster DAX calculations — fewer joins.	Slower performance — more joins required.
Easier to build and maintain.	Harder to maintain — changes require updating multiple tables.
Recommended for Power BI models.	Common in transactional databases (OLTP systems).
Best for: Power BI report development.	Best for: Source databases; should be flattened before loading into Power BI.

 **Best Practice — Flatten Snowflake to Star**

If you are connecting Power BI to a database that uses a Snowflake schema, it is recommended to merge (flatten) the related dimension tables into a single, combined dimension table before loading the data.

For example, instead of loading three separate tables — Products, Categories, and Brands — you would use Power Query to merge them into a single DimProduct table that contains all the descriptive columns. This converts the Snowflake into a Star and delivers better performance in Power BI.

Section 4

Creating & Managing Relationships

Step-by-Step in Power BI Desktop

Now that you understand the theory behind relationships, cardinality, and schemas, this section shows you exactly how to create and manage relationships inside Power BI Desktop. Relationships are managed in the Model View.

4.1 Accessing Model View

To view and manage relationships, you must switch to the Model View. On the left side panel of Power BI Desktop, click the diagram/network icon (the third icon from the top). The screen will switch to show all loaded tables as boxes, with lines connecting tables that already have relationships defined.

4.2 Automatic Relationship Detection

When you load multiple tables into Power BI, it attempts to automatically detect relationships by looking for columns with matching names and compatible data types. For example, if both a Sales table and a Products table have a column named 'Product ID', Power BI will usually create a relationship between them automatically.

Always Verify Auto-Detected Relationships

While automatic detection is a useful starting point, it is not always correct. Power BI may:

- Create a relationship using the wrong columns if multiple columns have similar names.
- Detect the wrong cardinality (e.g., Many-to-Many instead of One-to-Many).
- Miss a relationship entirely if column names do not match exactly.

Always switch to Model View after loading data and manually verify that every relationship is correct before building any visuals.

4.3 Creating a Relationship Manually

If Power BI has not automatically detected a relationship, or if you need to create one between columns with different names, you can create it manually using one of two methods.

Method 1 — Drag and Drop in Model View

This is the quickest and most visual method. Follow these steps:

1. Switch to **Model View** using the left panel icon.
2. Locate the two tables you want to connect. If they are not visible, scroll the canvas or use **Fit to Screen** in the View ribbon.
3. In the first table's box, identify the **key column** you want to use (e.g., 'Customer ID' in the Customers table).
4. Click and hold on that column name, then drag it across to the matching column in the second table (e.g., 'Customer ID' in the Orders table).
5. Release the mouse. Power BI will create the relationship and draw a line between the two tables.
6. Verify the cardinality symbols (1 and *) on the relationship line. Double-click the line to open the **Edit Relationship** dialog and confirm all settings.

Method 2 — Using the Manage Relationships Dialog

This method provides more control and is useful when you need to review all relationships in one place.

7. On the **Home** ribbon, click **Manage Relationships**. Alternatively, on the **Modeling** tab, click **Manage Relationships**.
8. The Manage Relationships dialog opens, showing a list of all existing relationships in the model.
9. Click **New...** to create a new relationship.
10. In the Create Relationship dialog, use the two dropdown menus to select the two tables.
11. Click on the key column in the top table's preview grid, then click the matching key column in the bottom table's preview grid.
12. Verify the **Cardinality** and **Cross filter direction** settings. Adjust if needed.
13. Click **OK** to save the relationship.

4.4 Editing and Deleting Relationships

Editing an Existing Relationship

To modify an existing relationship — for example, to change the cardinality or cross filter direction — double-click the relationship line in Model View. This opens the Edit Relationship dialog, where you can adjust all settings.

Deleting a Relationship

To delete a relationship, right-click on the relationship line in Model View and select 'Delete'. Alternatively, open Manage Relationships from the Home ribbon, select the relationship in the list, and click 'Delete'. Always confirm before deleting, as this action will affect all visuals that depend on the relationship.

Caution When Deleting Relationships

Deleting a relationship will immediately break any visuals, calculated columns, or measures that depend on it. If your report suddenly shows blank values or incorrect totals after a change, check the Model View first to confirm all required relationships are still intact.

ETHANS
CLOUD & AI ACADEMY



Section 5

Date Table Creation

The Foundation of Time-Based Analysis in Power BI

If you want to analyze your data over time — comparing sales this month versus last month, year-to-date totals, quarterly trends — you need a proper Date Table in your data model. A Date Table is not optional for time intelligence in Power BI; it is a requirement. This section explains why a Date Table is essential and how to create one correctly.

5.1 Why Do You Need a Dedicated Date Table?

Your Fact table likely already contains a date column — for example, an 'Order Date' column in a Sales table. You might wonder: why create a separate Date table? Why not just use the date column that already exists in the data?

 **Analogy — Why a Dedicated Date Table?**

Imagine your Sales table has 500,000 transaction rows, each with an Order Date. If someone asks 'What was total revenue in Q3?', Power BI would need to scan all 500,000 rows, check each date, determine which quarter it falls in, and then aggregate.

Now imagine a separate Date table with only 365 rows (one per day of the year), where each row already has columns like Day, Month Name, Quarter, Financial Year, Week Number, and Is Weekend — all pre-calculated.

Power BI can filter the Date table to 'Q3' in an instant (just a few rows), then use the relationship to retrieve only the matching Sales rows. This is dramatically faster and allows for complex time-based calculations that are simply not possible without a dedicated Date table.

Additionally, Power BI's built-in time intelligence DAX functions (like SAMEPERIODLASTYEAR, DATESYTD, DATEADD) only work correctly when a properly marked Date table exists in the model.

Without a Date Table	With a Dedicated Date Table
Cannot use DAX time intelligence functions.	Full access to all time intelligence functions (YTD, MTD, QTD, PY, etc.).
No pre-calculated time attributes (Quarter, Week, Is Holiday).	Rich time attributes available instantly: Day of Week, Fiscal Year, Month Name, etc.

Without a Date Table	With a Dedicated Date Table
Slower performance — scanning date column in large Fact tables.	Faster filtering — small Date table acts as a lookup for the large Fact table.
No consistent calendar available for multiple Fact tables.	Single Date table can be related to multiple Fact tables simultaneously.

5.2 What a Date Table Should Contain

A well-designed Date table should have one row for every calendar day within the range of your data — from the earliest date in your data to the latest (or the end of the current financial year). It should contain a continuous, unbroken sequence of dates — no gaps. Each row must also include a variety of pre-calculated time attributes that analysts and report users will need for filtering and grouping.

Column Name	Example Value	Purpose
Date	15/03/2025	The primary key — one unique date per row. Used to join with Fact tables.
Day	15	The day number within the month.
Day Name	Saturday	The full name of the day of the week.
Day of Week Number	7	Numeric day position (1=Monday to 7=Sunday) — used for sorting.
Week Number	11	ISO week number of the year (1–53).
Month Number	3	Numeric month (1–12) — used for sorting months correctly.
Month Name	March	Full month name — used for display in visuals.
Month Short	Mar	Abbreviated month name — used in charts with limited space.
Quarter	Q1	Quarter label (Q1, Q2, Q3, Q4).
Quarter Number	1	Numeric quarter (1–4) — used for sorting.
Year	2025	The calendar year.
Year-Month	2025-03	Combined Year and Month for trend charts.
Financial Year	FY2025-26	Financial / fiscal year (configure based on your organization's year-end).
Is Weekend	TRUE / FALSE	Boolean flag — useful for filtering business days vs weekends.
Is Holiday	TRUE / FALSE	Boolean flag — mark public holidays for operational reporting.

5.3 Creating a Date Table in Power BI

There are several methods to create a Date table in Power BI. The two most commonly used approaches for beginners and intermediate users are: using the CALENDARAUTO DAX function and using Power Query (M language). A third approach is to maintain an Excel-based Date table and import it.

Method 1 — Using DAX (Recommended for Beginners)

DAX provides built-in functions specifically designed to create date tables. This method creates the Date table entirely within Power BI — no external file is needed.

Steps to create a Date table using DAX:

14. In Power BI Desktop, go to the **Modeling** tab in the ribbon.
15. Click **New Table**. A formula bar will appear at the top.
16. Type the following DAX formula to create the Date table:

DAX — Create Date Table using CALENDARAUTO

```
DimDate =
VAR MinDate = MIN(FactSales[Order Date])
VAR MaxDate = MAX(FactSales[Order Date])
RETURN
ADDCOLUMNS(
    CALENDAR(MinDate, MaxDate),
    "Year",    YEAR([Date]),
    "Month Number", MONTH([Date]),
    "Month Name",    FORMAT([Date], "MMMM"),
    "Month Short",    FORMAT([Date], "MMM"),
    "Quarter",    "Q" & QUARTER([Date]),
    "Quarter Number",QUARTER([Date]),
    "Day",    DAY([Date]),
    "Day Name",    FORMAT([Date], "DDDD"),
    "Day of Week", WEEKDAY([Date], 2),
    "Week Number", WEEKNUM([Date], 2),
    "Year-Month",    FORMAT([Date], "YYYY-MM"),
    "Is Weekend",    IF(WEEKDAY([Date],2) >= 6, TRUE(), FALSE())
)
```

Code Block 5.1 — DAX formula to create a DimDate table with common time attributes

17. Press **Enter** or click the checkmark. Power BI will generate the Date table and it will appear in the Fields Pane.
18. Select the new **DimDate** table in the Fields Pane or Data View.
19. Go to the **Modeling** tab and click **Mark as Date Table**. Select the 'Date' column as the primary date column.

Method 2 — Using Power Query (M Language)

Alternatively, a Date table can be created using Power Query (the data transformation tool within Power BI). This approach is preferred by data engineers because it integrates with the existing data loading process and can be parameterized to automatically adjust the date range each time data is refreshed.

20. In Power BI Desktop, go to the **Home** tab and click **Transform Data** to open Power Query Editor.
21. In Power Query, go to **Home > New Source > Blank Query**.
22. Click on the new query in the left panel. Then click the **Advanced Editor** button.
23. Delete the existing content and paste the M code shown below:

```
Power Query M — Create Date Table

let
    StartDate = #date(2020, 1, 1),
    EndDate = #date(2025, 12, 31),
    DateList = List.Dates(StartDate, Duration.Days(EndDate - StartDate) + 1,
        #duration(1,0,0,0)),
    DataTable = Table.FromList(DateList, Splitter.SplitByNothing(),
        {"Date"}, null, ExtraValues.Error),
    AddYear = Table.AddColumn(DataTable, "Year",
        each Date.Year([Date]), Int64.Type),
    AddMonth = Table.AddColumn(AddYear, "Month Number",
        each Date.Month([Date]), Int64.Type),
    AddMonthName = Table.AddColumn(AddMonth, "Month Name",
        each Date.ToText([Date], "MMMM"), type text),
    AddQtr = Table.AddColumn(AddMonthName, "Quarter",
        each "Q" & Text.From(Date.QuarterOfYear([Date])), type text)
in
    AddQtr
```

Code Block 5.2 — Power Query M formula to generate a Date table (abbreviated)

24. Click **Done**. Rename the query to 'DimDate' by right-clicking it in the Queries panel.
25. Click **Close & Apply** to load the Date table into the model.

5.4 Connecting the Date Table to Your Fact Tables

Once the Date table is created and marked, you must connect it to every Fact table that contains a date column. This is done using a One-to-Many relationship, where the Date table is on the 'one' side (each date appears once) and the Fact table is on the 'many' side (the same date can appear in many transactions).

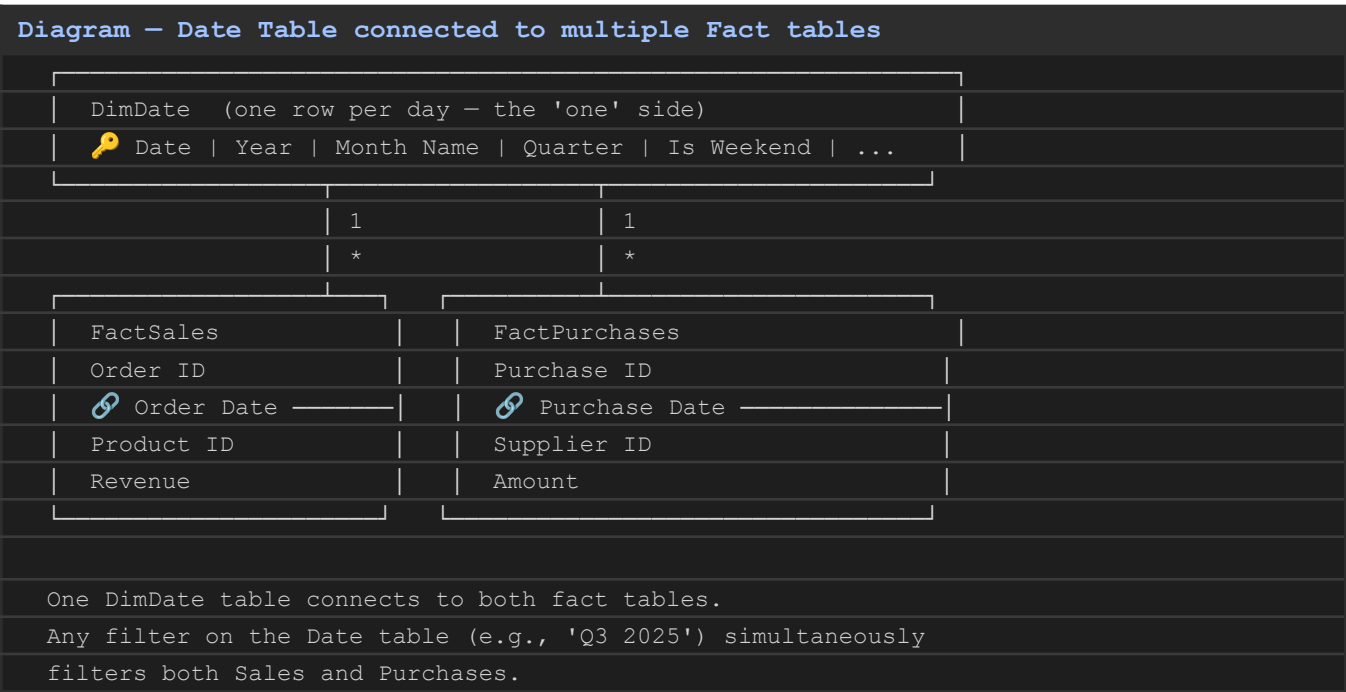


Diagram 5.1 — A single Date table connected to multiple Fact tables

 **Pro Tip**

Always use one shared Date table connected to all your Fact tables — never create separate date tables for each Fact table. A single Date table ensures that all your time-based visuals and calculations are synchronized. When a user selects 'March 2025' from a slicer, it will consistently filter all visuals across the report.

Section 6

Hierarchies in Power BI

Organizing Data for Drill-Down Analysis

A Hierarchy in Power BI is an organized, layered grouping of columns that allows report users to navigate from a broad summary level down to a detailed level by drilling into the data. Hierarchies are one of the features that make Power BI reports genuinely interactive and self-service.

6.1 What is a Hierarchy?

Hierarchy	An ordered sequence of columns arranged from the broadest (highest) level to the most specific (lowest) level, where each level contains the data of the level above it. Hierarchies enable drill-down and drill-up interactions in visuals — allowing users to explore data from a summary view all the way down to individual record detail.
------------------	--

 Analogy — Understanding Hierarchies
Consider how a large organization is structured:
Country → State → City → Store → Department
India is a Country that contains multiple States. Each State contains multiple Cities. Each City may have multiple Stores. Each Store has multiple Departments.
When looking at a sales chart at the Country level, you see total sales for India. If you drill down, you see sales broken by State (Maharashtra, Gujarat, Rajasthan, etc.). Drilling down further shows City-level sales within each state, and so on.
This is a geographic hierarchy. Power BI lets you define this structure so that a single visual can navigate all five levels with a single click.

6.2 Common Hierarchy Types in Business Reporting

Hierarchy Name	Levels (Top to Bottom)	Used In
Date / Time Hierarchy	Year > Quarter > Month > Week > Day	Sales trends, financial reporting, attendance tracking
Geographic Hierarchy	Country > State > City > Postal Code	Sales by region, logistics, branch performance
Product Hierarchy	Category > Sub-Category > Brand > Product	Retail analysis, inventory, procurement
Organizational Hierarchy	Division > Department > Team > Employee	HR reporting, headcount, payroll
Account Hierarchy	Account Group > Account Type > Account > Sub-Account	Financial statements, GL reporting

6.3 Creating a Hierarchy in Power BI

Hierarchies are created in the Fields Pane of Power BI Desktop. The process involves selecting the column that will form the top level of the hierarchy and then adding lower-level columns to it in the correct order.

Step-by-Step: Creating a Product Hierarchy

The following example creates a Product Hierarchy with three levels: Category > Sub-Category > Product Name.

26. In the **Fields Pane** on the right, expand the **DimProduct** table to see its columns.
27. Right-click on the **Category** column (this will be the top level of the hierarchy).
28. From the context menu, select **Create hierarchy**. A new item named 'Category Hierarchy' will appear under the table in the Fields Pane.
29. Right-click on the **Sub-Category** column and select **Add to hierarchy > Category Hierarchy**. This adds Sub-Category as the second level.
30. Repeat for the **Product Name** column — right-click and select **Add to hierarchy > Category Hierarchy**.
31. The Fields Pane will now show the hierarchy with all three levels nested in order: Category > Sub-Category > Product Name. You can rename the hierarchy by double-clicking it (e.g., rename to **Product Hierarchy**).

6.4 Using Drill-Down with Hierarchies in Visuals

Once a hierarchy is created, it becomes available to use in any chart or visual. When you add a hierarchy to a visual's Axis field, Power BI automatically enables drill-down functionality — allowing report users to click through the levels of the hierarchy interactively.

Drill-Down Controls

When a hierarchy is active in a visual, a set of drill controls appears at the top of the visual. These controls allow you to navigate between hierarchy levels.

Control	Icon Description	Action
Drill Down	Arrow pointing down	Click a specific data point to see the next level for that item only (e.g., click 'Electronics' to see its sub-categories).
Drill Up	Arrow pointing up	Go back to the previous, broader level.
Expand All	Double downward arrows	Expand all items to the next level simultaneously (e.g., show sub-categories for all categories at once).
Next Level	Single downward fork arrow	Move all data to the next level without filtering (e.g., go from Category to Sub-Category for all).

Professional Example — Using Drill-Down in a Sales Report

A Regional Sales Manager opens the monthly sales report in Power BI. The main chart shows total sales by Country — India, UAE, and Singapore.

She notices India's sales are significantly higher than last month. She clicks the Drill Down icon, then clicks on 'India' in the chart. The chart immediately updates to show sales broken down by State: Maharashtra, Gujarat, Karnataka, Tamil Nadu.

She notices Maharashtra has an unusual spike. She drills down again — now she sees City-level data within Maharashtra. She finds that Mumbai alone accounts for 68% of the state's sales — driven by a new corporate client. The hierarchy allowed her to go from a country-level summary to a city-level insight in three clicks, without needing to build separate reports for each level.

Pro Tip

The Date Hierarchy (Year > Quarter > Month > Day) is automatically created by Power BI for every date column you load. You do not need to create it manually. However, if you want a custom Date Hierarchy (e.g., that uses your Financial Year definition rather than the calendar year), create a manual hierarchy in your DimDate table instead of using the auto-generated one.

Section 7

Practice Exercise

Applying Data Modeling Concepts in Power BI

7.1 Exercise Overview

This practice exercise is designed to reinforce everything covered in this chapter. You will build a complete star-schema data model from multiple tables, define all relationships correctly, create a Date table, and set up hierarchies — all using the practice data files provided by your instructor.

 **Before You Begin**

Your instructor will provide the practice Excel file: 'DataModeling_Practice.xlsx'. This file contains the following sheets:

- FactSales — Transaction-level sales records (Order ID, Date, Product ID, Customer ID, Store ID, Qty, Revenue)
- DimProduct — Product master (Product ID, Product Name, Sub-Category, Category)
- DimCustomer — Customer master (Customer ID, Name, City, State, Segment)
- DimStore — Store master (Store ID, Store Name, City, Region)

Load all four sheets into Power BI Desktop before beginning the exercises.

7.2 Exercise Instructions

Exercise 1 — Verify and Correct Relationships

After loading all four sheets, go to Model View and perform the following:

- Identify which relationships Power BI has automatically created. List them by writing: **Table A [Column] → Table B [Column]**.
- For each relationship, verify the cardinality. Is it correctly set to One-to-Many (1:*)? Are the '1' and '*' symbols on the correct sides?
- Check whether all three dimension tables (DimProduct, DimCustomer, DimStore) are connected to FactSales. If any connection is missing, create it manually using the drag-and-drop method.
- Verify that all relationships use Single cross filter direction.

✓ Expected Result — Exercise 1

After completing Exercise 1, your Model View should show the FactSales table at the center, connected to all three dimension tables — forming a Star Schema. Each connection should be a One-to-Many relationship (1:*) with Single cross filter direction.

Exercise 2 — Create and Connect a Date Table

Follow these steps to add a proper Date table to your model:

32. Go to the **Modeling** tab and click **New Table**.
33. Enter the DAX CALENDARAUTO formula covered in Section 5.3 (Method 1) to create the DimDate table.
34. Add the following columns to the DAX formula: Year, Month Number, Month Name, Quarter, Quarter Number, Day Name, Is Weekend.
35. Mark the DimDate table as the **Date Table** using Modeling > Mark as Date Table. Select the 'Date' column.
36. In Model View, connect **DimDate[Date]** to **FactSales[Order Date]** using a One-to-Many relationship.

Exercise 3 — Create Hierarchies

Create the following two hierarchies in your model:

- **Product Hierarchy** (in DimProduct): Category > Sub-Category > Product Name
- **Geography Hierarchy** (in DimCustomer): State > City > Customer Name

After creating both hierarchies, use each one in a bar chart visual to confirm the drill-down controls appear and function correctly.

Exercise 4 — Test Your Model

Go to Report View and create the following three visuals to validate your data model is working correctly:

37. **Bar Chart**: Total Revenue by Category (using Product Hierarchy). Drill down to Sub-Category level.
38. **Line Chart**: Total Revenue by Month Name from DimDate. Verify that months appear in the correct calendar order (Jan, Feb, Mar...) — not alphabetical order. (Tip: Sort Month Name by Month Number.)
39. **Card Visual**: Show Total Revenue. Then add a Slicer for Year (from DimDate). Confirm that selecting a year correctly filters the Card visual.

📌 Exercise Completion Checklist

Before moving to the next chapter, confirm you can do the following without assistance:

- ✓ Explain the difference between One-to-Many, Many-to-Many, and One-to-One relationships

- | |
|--|
| ✓ Identify cardinality symbols (1 and *) in Model View |
| ✓ Explain the difference between Single and Both cross filter directions |
| ✓ Describe the structure of a Star Schema vs a Snowflake Schema |
| ✓ Create a relationship manually using drag-and-drop or Manage Relationships |
| ✓ Create a Date table using DAX and mark it as a Date Table |
| ✓ Create a hierarchy and use drill-down in a visual |

ETHANS
CLOUD & AI ACADEMY



Chapter Summary

In this chapter, you have covered the complete foundation of data modeling in Power BI. Here is a recap of every key concept:

- **Relationships** are the defined connections between tables in a data model, based on a shared key column. They allow Power BI to combine and filter data across multiple tables.
- The three types of relationships are: **One-to-Many (1:*)** — the most common type; **Many-to-Many (*:*)** — best resolved with a bridge table; and **One-to-One (1:1)** — used for split entity tables.
- **Cardinality** describes how many rows in one table match rows in another. It is represented by '1' (unique side) and '*' (many side) on relationship lines in Model View.
- **Cross Filter Direction** controls how filter selections travel between tables. **Single** direction (default, recommended) filters from the '1' side to the '*' side. **Both** directions allow bidirectional filtering but must be used with caution.
- A **Star Schema** places a central Fact table surrounded by directly connected Dimension tables. It is the recommended design for Power BI for maximum performance and simplicity.
- A **Snowflake Schema** normalizes Dimension tables into sub-dimensions, creating more tables and joins. It should be flattened into a Star Schema before use in Power BI.
- A **Fact Table** stores measurable business events (transactions, orders). A **Dimension Table** stores descriptive context (products, customers, dates).
- A dedicated **Date Table** is required for time intelligence calculations in Power BI. It must have one row per day, no gaps, and be marked as a Date Table. It connects to all Fact tables via a One-to-Many relationship.
- **Hierarchies** organize columns from broad to specific (e.g., Year > Quarter > Month > Day). They enable drill-down and drill-up navigation in visuals, making reports interactive and exploratory.